Лабораторная работа №19
Разработка модулей программного обеспечения

1 Цель работы

1.1 Изучить процесс создания и применения библиотек в программах на языке C#.

2 Литература

2.1 Ашарина, И. В. Объектно-ориентированное программирование в C++: лекции и упражнения. Учебное пособие для вузов. – 2-е изд., перераб. и доп. Москва : Горячая Линия–Телеком, 2017. – URL: https://ibooks.ru/bookshelf/359752/reading. – Режим доступа: для зарегистрир. пользователей. – Текст : электронный. – гл.11.

3 Подготовка к работе

3.1 Повторить теоретический материал (см. п.2).

3.2 Изучить описание лабораторной работы.

4 Основное оборудование

4.1 Персональный компьютер.

5 Задание

5.1 Создать новый проект Приложение WindowsForms

Добавить в решение новый проект **Библиотека классов Windows Forms** с именем WinFormsHelpers.

В проекте **WindowsForms** в контекстном меню обозревателя решений выбрать: **Добавить – Ссылка на проект** и установить галочку напротив проекта **WinFormsHelpers**

5.2 В проекте WinFormsHelpers создать новый статический класс NumericHelper. В классе реализовать следующие методы:

- IsInRange(int value, int min, int max) – проверяет, что значение в диапазоне
- ClampValue(int value, int min, int max) – возвращает значение внутри диапазона, если значение меньше минимального, возвращает минимальное, если больше максимального - максимальное

Протестировать созданные методы в главном приложении

5.3 В проекте WinFormsHelpers создать новый статический класс MessageBoxExtensions. В классе реализовать следующие методы:

- ShowInfo(string message)
- ShowQuestion(string question)
- ShowError(string error)
- ShowError(Exception ex)

Каждый из методов должен отображать пользователю диалоговое окно MessageBox с переданным сообщением, заранее настроенным заголовком, кнопками и иконкой.

Протестировать созданные методы в главном приложении

5.4 В проекте WinFormsHelpers создать новый статический класс TextBoxExtensions. В классе реализовать следующие методы или свойства:

- IsEmpty – возвращает true, если поле пустое
- HighlightError – меняет фон или границу поля ввода для уведомления о некорректном вводе
- ClearError – возвращает поле ввода в исходное состояние после подсветки.

Протестировать созданные методы в главном приложении

5.5 Изменить все созданные методы на методы расширения, протестировать их работу в приложении.

6 Порядок выполнения работы

6.1 Используя Microsoft Visual Studio, создать проект C# и выполнить задания из п.5.

6.2 Ответить на контрольные вопросы.

7 Содержание отчета

7.1 Титульный лист
7.2 Цель работы
7.3 Ответы на контрольные вопросы
7.4 Вывод

8 Контрольные вопросы

8.1 Зачем нужны библиотеки?
8.2 Как подключить библиотеку классов к проекту?

9 Приложение

Методы расширения (Extension Methods) позволяют добавлять новые методы к существующим типам без изменения их исходного кода и наследования.

Например:

Если необходимо создать метод, который проверяет, начинается ли строка с заглавной буквы.

В обычном случае мы реализуем его так:

```
public static bool StartsWithUpper(string text)
{
```

```
    return text.Length > 0 && char.IsUpper(text[0]);
}
```

Для использования данного метода необходимо вызвать его так:

```
bool result = StartsWithUpper("Привет");
```

Если же реализовать ту же задачу при помощи метода расширения, в метод к параметру добавляется ключевое слово this. Кроме того, метод расширения обязательно должен быть статическим и располагаться в статическом классе:

```
public static class StringExtensions
{
    public static bool StartsWithUpper(this string text)
    {
        return text.Length > 0 && char.IsUpper(text[0]);
    }
}
```

Использование метода расширения выглядит несколько иначе и более гибко:

```
bool result = "Привет".StartsWithUpper();
```

В новых версиях C# появилась конструкция extension, которая позволяет объявлять расширения несколько иначе:

```
public static class StringExtensions
{
    extension(string text)
    {
        public bool StartsWithUpper()
        {
            return text.Length > 0 &&
char.IsUpper(text[0]);
        }
    }
}
```

Такой способ позволяет также создавать свойства расширения и статические расширения типов:

```
public static class StringExtensions
```

```
{
    extension(string text)
    {
        public bool IsLong => text.Length > 10;
    }
}
```

```
string text = "Привет, мир!";

bool result = text.IsLong;
```

Статические расширения позволяют добавлять методы и свойства, которые вызываются через имя самого типа, а не через его экземпляр.

```
public static class StringExtensions
{
    extension(string)
    {
        public static string Repeat(char symbol, int
count)
        {
            return new string(symbol, count);
        }
    }
}
```

```
string text = string.Repeat('*', 5);
```